

第 3 章 ROS通信机制进阶

上一章内容，主要介绍了ROS通信的实现，内容偏向于粗粒度的通信框架的讲解，没有详细介绍涉及的API，也没有封装代码，鉴于此，本章主要内容如下：

- ROS常用API介绍；
- ROS中自定义头文件与源文件的使用。

预期达成的学习目标：

- 熟练掌握ROS常用API；
- 掌握ROS中自定义头文件与源文件的配置。

3.1 常用API

首先，建议参考官方API文档或参考源码：

- ROS节点的初始化相关API；
- NodeHandle 的基本使用相关API；
- 话题的发布方，订阅方对象相关API；
- 服务的服务端，客户端对象相关API；
- 时间相关API；
- 日志输出相关API。

参数服务器相关API在第二章已经有详细介绍和应用，在此不再赘述。

另请参考：

- <http://wiki.ros.org/APIs>
- <https://docs.ros.org/en/api/roscpp/html/>

3.1.1 初始化

C++

初始化

```
1  /** @brief ROS初始化函数。
2   *
3   * 该函数可以解析并使用节点启动时传入的参数(通过参数设置节点名称、命名空间...)
4   *
5   * 该函数有多个重载版本，如果使用NodeHandle建议调用该版本。
6   *
7   * \param argc 参数个数
8   * \param argv 参数列表
```

```

9      * \param name 节点名称, 需要保证其唯一性, 不允许包含命名空间
10     * \param options 节点启动选项, 被封装进了ros::init_options
11     *
12     */
13 void init(int &argc, char **argv, const std::string& name, uint32_t options =
14     0);

```

Python

初始化

```

1 def init_node(name, argv=None, anonymous=False, log_level=None,
2   disable_rostime=False, disable_rosout=False, disable_signals=False,
3   xmlrpc_port=0, tcpport=0):
4     """
5     在ROS master中注册节点
6
7     @param name: 节点名称, 必须保证节点名称唯一, 节点名称中不能使用命名空间(不能包
8     含 '/')
9     @type name: str
10
11     @param anonymous: 取值为 true 时, 为节点名称后缀随机编号
12     @type anonymous: bool
13     """

```

3.1.2 话题与服务相关对象

C++

在 roscpp 中, 话题和服务的相关对象一般由 NodeHandle 创建。

NodeHandle有一个重要作用是可以用于设置命名空间, 这是后期的重点, 但是本章暂不介绍。

1.发布对象

对象获取:

```

1 /**
2  * \brief 根据话题生成发布对象
3  *
4  * 在 ROS master 注册并返回一个发布者对象, 该对象可以发布消息
5  *
6  * 使用示例如下:
7  *
8  *   ros::Publisher pub = handle.advertise<std_msgs::Empty>("my_topic", 1);
9  *
10 * \param topic 发布消息使用的话题
11 *

```

```

12  * \param queue_size 等待发送给订阅者的最大消息数量
13  *
14  * \param latch (optional) 如果为 true,该话题发布的最后一条消息将被保存, 并且后期
    当有订阅者连接时会将该消息发送给订阅者
15  *
16  * \return 调用成功时, 会返回一个发布对象
17  *
18  *
19  */
20  template <class M>
21  Publisher advertise(const std::string& topic, uint32_t queue_size, bool latch
    = false)

```

消息发布函数:

```

1  /**
2  * 发布消息
3  */
4  template <typename M>
5  void publish(const M& message) const

```

2.订阅对象

对象获取:

```

1  /**
2  * \brief 生成某个话题的订阅对象
3  *
4  * 该函数将根据给定的话题在ROS master 注册, 并自动连接相同主题的发布方, 每接收到
    一条消息, 都会调用回调
5  * 函数, 并且传入该消息的共享指针, 该消息不能被修改, 因为可能其他订阅对象也会使用
    该消息。
6  *
7  * 使用示例如下:
8
9  void callback(const std_msgs::Empty::ConstPtr& message)
10 {
11 }
12
13 ros::Subscriber sub = handle.subscribe("my_topic", 1, callback);
14
15 *
16 * \param M [template] M 是指消息类型
17 * \param topic 订阅的话题
18 * \param queue_size 消息队列长度, 超出长度时, 头部的消息将被弃用
19 * \param fp 当订阅到一条消息时, 需要执行的回调函数
20 * \return 调用成功时, 返回一个订阅者对象, 失败时, 返回空对象
21 *
22
23 void callback(const std_msgs::Empty::ConstPtr& message){...}
24 ros::NodeHandle nodeHandle;
25 ros::Subscriber sub = nodeHandle.subscribe("my_topic", 1, callback);

```

```

26 | if (sub) // Enter if subscriber is valid
27 | {
28 | ...
29 | }
30 |
31 | */
32 | template<class M>
33 | Subscriber subscribe(const std::string& topic, uint32_t queue_size, void(*fp)
   | (const boost::shared_ptr<M const>&), const TransportHints& transport_hints =
   | TransportHints())

```

3.服务对象

对象获取:

```

1 | /**
2 |  * \brief 生成服务端对象
3 |  *
4 |  * 该函数可以连接到 ROS master, 并提供一个具有给定名称的服务对象。
5 |  *
6 |  * 使用示例如下:
7 |  \verbatim
8 |  bool callback(std_srvs::Empty& request, std_srvs::Empty& response)
9 |  {
10 |  return true;
11 |  }
12 |
13 |  ros::ServiceServer service = handle.advertiseService("my_service", callback);
14 |  \endverbatim
15 |  *
16 |  * \param service 服务的主题名称
17 |  * \param srv_func 接收到请求时, 需要处理请求的回调函数
18 |  * \return 请求成功时返回服务对象, 否则返回空对象:
19 |  \verbatim
20 |  bool Foo::callback(std_srvs::Empty& request, std_srvs::Empty& response)
21 |  {
22 |  return true;
23 |  }
24 |  ros::NodeHandle nodeHandle;
25 |  Foo foo_object;
26 |  ros::ServiceServer service = nodeHandle.advertiseService("my_service",
   |  callback);
27 |  if (service) // Enter if advertised service is valid
28 |  {
29 |  ...
30 |  }
31 |  \endverbatim
32 |
33 |  */
34 | template<class MReq, class MRes>
35 | ServiceServer advertiseService(const std::string& service, bool(*srv_func)
   | (MReq&, MRes&))

```

4.客户端对象

对象获取:

```
1  /**
2   * @brief 创建一个服务客户端对象
3   *
4   * 当清除最后一个连接的引用句柄时，连接将被关闭。
5   *
6   * @param service_name 服务主题名称
7   */
8  template<class Service>
9  ServiceClient serviceClient(const std::string& service_name, bool persistent
= false,
10                             const M_string& header_values = M_string())
```

请求发送函数:

```
1  /**
2   * @brief 发送请求
3   * 返回值为 bool 类型，true，请求处理成功，false，处理失败。
4   */
5  template<class Service>
6  bool call(Service& service)
```

等待服务函数1:

```
1  /**
2   * ros::service::waitForService("addInts");
3   * \brief 等待服务可用，否则一直处于阻塞状态
4   * \param service_name 被"等待"的服务的话题名称
5   * \param timeout 等待最大时常，默认为 -1，可以永久等待直至节点关闭
6   * \return 成功返回 true，否则返回 false。
7   */
8  ROSCPP_DECL bool waitForService(const std::string& service_name, ros::Duration
timeout = ros::Duration(-1));
```

等待服务函数2:

```
1  /**
2   * client.waitForExistence();
3   * \brief 等待服务可用，否则一直处于阻塞状态
4   * \param timeout 等待最大时常，默认为 -1，可以永久等待直至节点关闭
5   * \return 成功返回 true，否则返回 false。
6   */
7  bool waitForExistence(ros::Duration timeout = ros::Duration(-1));
```

1.发布对象

对象获取:

```
1 class Publisher(Topic):
2     """
3     在ROS master注册为相关话题的发布方
4     """
5
6     def __init__(self, name, data_class, subscriber_listener=None,
7 tcp_nodelay=False, latch=False, headers=None, queue_size=None):
8         """
9         Constructor
10        @param name: 话题名称
11        @type name: str
12        @param data_class: 消息类型
13
14        @param latch: 如果为 true,该话题发布的最后一条消息将被保存, 并且后期当有
15        订阅者连接时会将该消息发送给订阅者
16        @type latch: bool
17
18        @param queue_size: 等待发送给订阅者的最大消息数量
19        @type queue_size: int
20
21        """
```

消息发布函数:

```
1 def publish(self, *args, **kws):
2     """
3     发布消息
4     """
```

2.订阅对象

对象获取:

```
1 class Subscriber(Topic):
2     """
3     类注册为指定主题的订阅者, 其中消息是给定类型的。
4     """
5
6     def __init__(self, name, data_class, callback=None, callback_args=None,
7 queue_size=None, buff_size=DEFAULT_BUFF_SIZE,
8 tcp_nodelay=False):
9         """
10        Constructor.
11
12        @param name: 话题名称
13        @type name: str
14        @param data_class: 消息类型
15        @type data_class: L{Message} class
16        @param callback: 处理订阅到的消息的回调函数
```

```

15         @type callback: fn(msg, cb_args)
16
17         @param queue_size: 消息队列长度，超出长度时，头部的消息将被弃用
18
19         """

```

3.服务对象

对象获取:

```

1 class Service(ServiceImpl):
2     """
3     声明一个ROS服务
4
5     使用示例::
6         s = Service('getmapservice', GetMap, get_map_handler)
7     """
8
9     def __init__(self, name, service_class, handler,
10                  buff_size=DEFAULT_BUFF_SIZE, error_handler=None):
11         """
12
13         @param name: 服务主题名称 ``str``
14         @param service_class: 服务消息类型
15
16         @param handler: 回调函数，处理请求数据，并返回响应数据
17
18         @type handler: fn(req)->resp
19
20         """

```

4.客户端对象

对象获取:

```

1 class ServiceProxy(_Service):
2     """
3     创建一个ROS服务的句柄
4
5     示例用法::
6         add_two_ints = ServiceProxy('add_two_ints', AddTwoInts)
7         resp = add_two_ints(1, 2)
8     """
9
10    def __init__(self, name, service_class, persistent=False, headers=None):
11        """
12        ctor.
13        @param name: 服务主题名称
14        @type name: str
15        @param service_class: 服务消息类型
16        @type service_class: Service class
17        """

```

请求发送函数:

```
1 def call(self, *args, **kws):
2     """
3     发送请求, 返回值为响应数据
4
5
6     """
```

等待服务函数:

```
1 def wait_for_service(service, timeout=None):
2     """
3     调用该函数时, 程序会处于阻塞状态直到服务可用
4     @param service: 被等待的服务话题名称
5     @type service: str
6     @param timeout: 超时时间
7     @type timeout: double|rospy.Duration
8     """
```

3.1.3 回旋函数

C++

在ROS程序中, 频繁的使用了 `ros::spin()` 和 `ros::spinOnce()` 两个回旋函数, 可以用于处理回调函数。

1.spinOnce()

```
1 /**
2  * \brief 处理一轮回调
3  *
4  * 一般应用场景:
5  *      在循环体内, 处理所有可用的回调函数
6  *
7  */
8 ROSCPP_DECL void spinOnce();
```

2.spin()

```
1 /**
2  * \brief 进入循环处理回调
3  */
4 ROSCPP_DECL void spin();
```

3.二者比较

相同点:二者都用于处理回调函数:

不同点:ros::spin() 是进入了循环执行回调函数, 而 ros::spinOnce() 只会执行一次回调函数(没有循环), 在 ros::spin() 后的语句不会执行到, 而 ros::spinOnce() 后的语句可以执行。

Python

```
1 def spin():
2     """
3     进入循环处理回调
4     """
```

3.1.4 时间

ROS中时间相关的API是极其常用, 比如:获取当前时刻、持续时间的设置、执行频率、休眠、定时器...都与时间相关。

C++

1.时刻

获取时刻, 或是设置指定时刻:

```
1 ros::init(argc,argv,"hello_time");
2 ros::NodeHandle nh;//必须创建句柄, 否则时间没有初始化, 导致后续API调用失败
3 ros::Time right_now = ros::Time::now();//将当前时刻封装成对象
4 ROS_INFO("当前时刻:%.2f",right_now.toSec());//获取距离 1970年01月01日 00:00:00
  的秒数
5 ROS_INFO("当前时刻:%d",right_now.sec);//获取距离 1970年01月01日 00:00:00 的秒数
6
7 ros::Time someTime(100,100000000);// 参数1:秒数 参数2:纳秒
8 ROS_INFO("时刻:%.2f",someTime.toSec()); //100.10
9 ros::Time someTime2(100.3);//直接传入 double 类型的秒数
10 ROS_INFO("时刻:%.2f",someTime2.toSec()); //100.30
```

2.持续时间

设置一个时间区间(间隔):

```
1 ROS_INFO("当前时刻:%.2f",ros::Time::now().toSec());
2 ros::Duration du(10);//持续10秒钟, 参数是double类型的, 以秒为单位
3 du.sleep();//按照指定的持续时间休眠
4 ROS_INFO("持续时间:%.2f",du.toSec());//将持续时间换算成秒
5 ROS_INFO("当前时刻:%.2f",ros::Time::now().toSec());
```

3.持续时间与时刻运算

为了方便使用, ROS中提供了时间与时刻的运算:

```

1 ROS_INFO("时间运算");
2 ros::Time now = ros::Time::now();
3 ros::Duration du1(10);
4 ros::Duration du2(20);
5 ROS_INFO("当前时刻:%.2f",now.toSec());
6 //1.time 与 duration 运算
7 ros::Time after_now = now + du1;
8 ros::Time before_now = now - du1;
9 ROS_INFO("当前时刻之后:%.2f",after_now.toSec());
10 ROS_INFO("当前时刻之前:%.2f",before_now.toSec());
11
12 //2.duration 之间相互运算
13 ros::Duration du3 = du1 + du2;
14 ros::Duration du4 = du1 - du2;
15 ROS_INFO("du3 = %.2f",du3.toSec());
16 ROS_INFO("du4 = %.2f",du4.toSec());
17 //PS: time 与 time 不可以运算
18 // ros::Time nn = now + before_now;//异常

```

4.设置运行频率

```

1 ros::Rate rate(1);//指定频率
2 while (true)
3 {
4     ROS_INFO("-----code-----");
5     rate.sleep();//休眠, 休眠时间 = 1 / 频率。
6 }

```

5.定时器

ROS 中内置了专门的定时器，可以实现与 `ros::Rate` 类似的效果：

```

1 ros::NodeHandle nh;//必须创建句柄，否则时间没有初始化，导致后续API调用失败
2
3 // ROS 定时器
4 /**
5  * \brief 创建一个定时器，按照指定频率调用回调函数。
6  *
7  * \param period 时间间隔
8  * \param callback 回调函数
9  * \param oneshot 如果设置为 true,只执行一次回调函数，设置为 false,就循环执行。
10  * \param autostart 如果为true，返回已经启动的定时器,设置为 false，需要手动启动。
11  */
12 //Timer createTimer(Duration period, const TimerCallback& callback, bool
oneshot = false,
13 //                    bool autostart = true) const;
14
15 // ros::Timer timer = nh.createTimer(ros::Duration(0.5),doSomething);
16 ros::Timer timer = nh.createTimer(ros::Duration(0.5),doSomething,true);//只
执行一次
17

```

```

18 // ros::Timer timer =
    nh.createTimer(ros::Duration(0.5),doSomething,false,false);//需要手动启动
19 // timer.start();
20 ros::spin(); //必须 spin

```

定时器的回调函数:

```

1 void doSomething(const ros::TimerEvent &event){
2     ROS_INFO("-----");
3     ROS_INFO("event:%s",std::to_string(event.current_real.toSec()).c_str());
4 }

```

#####

Python

1.时刻

获取时刻，或是设置指定时刻:

```

1 # 获取当前时刻
2 right_now = rospy.Time.now()
3 rospy.loginfo("当前时刻:%.2f",right_now.to_sec())
4 rospy.loginfo("当前时刻:%.2f",right_now.to_nsec())
5 # 自定义时刻
6 some_time1 = rospy.Time(1234.567891011)
7 some_time2 = rospy.Time(1234,567891011)
8 rospy.loginfo("设置时刻1:%.2f",some_time1.to_sec())
9 rospy.loginfo("设置时刻2:%.2f",some_time2.to_sec())
10
11 # 从时间创建对象
12 # some_time3 = rospy.Time.from_seconds(543.21)
13 some_time3 = rospy.Time.from_sec(543.21) # from_sec 替换了 from_seconds
14 rospy.loginfo("设置时刻3:%.2f",some_time3.to_sec())

```

2.持续时间

设置一个时间区间(间隔):

```

1 # 持续时间相关API
2 rospy.loginfo("持续时间测试开始.....")
3 du = rospy.Duration(3.3)
4 rospy.loginfo("du1 持续时间:%.2f",du.to_sec())
5 rospy.sleep(du) #休眠函数
6 rospy.loginfo("持续时间测试结束.....")

```

3.持续时间与时刻运算

为了方便使用，ROS中提供了时间与时刻的运算:

```

1  rospy.loginfo("时间运算")
2  now = rospy.Time.now()
3  du1 = rospy.Duration(10)
4  du2 = rospy.Duration(20)
5  rospy.loginfo("当前时刻:%.2f",now.to_sec())
6  before_now = now - du1
7  after_now = now + du1
8  dd = du1 + du2
9  # now = now + now #非法
10 rospy.loginfo("之前时刻:%.2f",before_now.to_sec())
11 rospy.loginfo("之后时刻:%.2f",after_now.to_sec())
12 rospy.loginfo("持续时间相加:%.2f",dd.to_sec())

```

4.设置运行频率

```

1  # 设置执行频率
2  rate = rospy.Rate(0.5)
3  while not rospy.is_shutdown():
4      rate.sleep() #休眠
5      rospy.loginfo("+++++")

```

5.定时器

ROS 中内置了专门的定时器，可以实现与 `ros::Rate` 类似的效果：

```

1  #定时器设置
2  """
3  def __init__(self, period, callback, oneshot=False, reset=False):
4      Constructor.
5      @param period: 回调函数的时间间隔
6      @type period: rospy.Duration
7      @param callback: 回调函数
8      @type callback: function taking rospy.TimerEvent
9      @param oneshot: 设置为True, 就只执行一次, 否则循环执行
10     @type oneshot: bool
11     @param reset: if True, timer is reset when rostime moved backward.
12     [default: False]
13     @type reset: bool
14     """
15     rospy.Timer(rospy.Duration(1),doMsg)
16     # rospy.Timer(rospy.Duration(1),doMsg,True) # 只执行一次
17     rospy.spin()

```

回调函数：

```

1  def doMsg(event):
2      rospy.loginfo("+++++")
3      rospy.loginfo("当前时刻:%s",str(event.current_real))

```

3.1.5 其他函数

在发布实现时，一般会循环发布消息，循环的判断条件一般由节点状态来控制，C++中可以通过 `ros::ok()` 来判断节点状态是否正常，而 `python` 中则通过 `rospy.is_shutdown()` 来实现判断，导致节点退出的原因主要有如下几种：

- 节点接收到了关闭信息，比如常用的 `ctrl + c` 快捷键就是关闭节点的信号；
- 同名节点启动，导致现有节点退出；
- 程序中的其他部分调用了节点关闭相关的API(C++中是`ros::shutdown()`，python中是`rospy.signal_shutdown()`)

另外，日志相关的函数也是极其常用的，在ROS中日志被划分成如下级别：

- **DEBUG(调试)**:只在调试时使用，此类消息不会输出到控制台；
 - **INFO(信息)**:标准消息，一般用于说明系统内正在执行的操作；
 - **WARN(警告)**:提醒一些异常情况，但程序仍然可以执行；
 - **ERROR(错误)**:提示错误信息，此类错误会影响程序运行；
 - **FATAL(严重错误)**:此类错误将阻止节点继续运行。
-

C++

1.节点状态判断

```
1  /** \brief 检查节点是否已经退出
2   *
3   *  ros::shutdown() 被调用且执行完毕后，该函数将会返回 false
4   *
5   *  \return true 如果节点还健在，false 如果节点已经火化了。
6   */
7  bool ok();
```

2.节点关闭函数

```
1  /**
2   *   关闭节点
3   */
4  void shutdown();
```

3.日志函数

使用示例

```
1  ROS_DEBUG("hello,DEBUG"); //不会输出
2  ROS_INFO("hello,INFO"); //默认白色字体
3  ROS_WARN("Hello,WARN"); //默认黄色字体
4  ROS_ERROR("hello,ERROR");//默认红色字体
5  ROS_FATAL("hello,FATAL");//默认红色字体
```

Python

1.节点状态判断

```

1 def is_shutdown():
2     """
3     @return: True 如果节点已经被关闭
4     @rtype: bool
5     """

```

2. 节点关闭函数

```

1 def signal_shutdown(reason):
2     """
3     关闭节点
4     @param reason: 节点关闭的原因，是一个字符串
5     @type reason: str
6     """
7     Copy
8     def on_shutdown(h):
9         """
10        节点被关闭时调用的函数
11        @param h: 关闭时调用的回调函数，此函数无参
12        @type h: fn()
13        """

```

3. 日志函数

使用示例

```

1 rospy.logdebug("hello, debug") #不会输出
2 rospy.loginfo("hello, info") #默认白色字体
3 rospy.logwarn("hello, warn") #默认黄色字体
4 rospy.logerr("hello, error") #默认红色字体
5 rospy.logfatal("hello, fatal") #默认红色字体

```

3.2 ROS中的头文件与源文件

本节主要介绍ROS的C++实现中，如何使用头文件与源文件的方式封装代码，具体内容如下：

1. 设置头文件，可执行文件作为源文件；
2. 分别设置头文件，源文件与可执行文件。

在ROS中关于头文件的使用，核心内容在于CMakeLists.txt文件的配置，不同的封装方式，配置上也有差异。

3.2.1 自定义头文件调用

需求:设计头文件，可执行文件本身作为源文件。

流程:

1. 编写头文件；
2. 编写可执行文件(同时也是源文件)；
3. 编辑配置文件并执行。

1.头文件

在功能包下的 include/功能包名 目录下新建头文件: hello.h, 示例内容如下:

```
1  #ifndef _HELLO_H
2  #define _HELLO_H
3
4  namespace hello_ns{
5
6  class HelloPub {
7
8  public:
9      void run();
10 };
11
12 }
13
14 #endif
```

注意:

在 VScode 中, 为了后续包含头文件时不抛出异常, 请配置 .vscode 下 c_cpp_properties.json 的 includepath属性

```
1  | "/home/用户/工作空间/src/功能包/include/**"
```

2.可执行文件

在 src 目录下新建文件:hello.cpp, 示例内容如下:

```
1  #include "ros/ros.h"
2  #include "test_head/hello.h"
3
4  namespace hello_ns {
5
6  void HelloPub::run(){
7      ROS_INFO("自定义头文件的使用....");
8  }
9
10 }
11
12 int main(int argc, char *argv[])
13 {
14     setlocale(LC_ALL, "");
15     ros::init(argc, argv, "test_head_node");
16     hello_ns::HelloPub helloPub;
17     helloPub.run();
18     return 0;
19 }
```

3.配置文件

配置CMakeLists.txt文件，头文件相关配置如下：

```
1 | include_directories(  
2 | include  
3 |     ${catkin_INCLUDE_DIRS}  
4 | )
```

可执行配置文件配置方式与之前一致：

```
1 | add_executable(hello src/hello.cpp)  
2 |  
3 | add_dependencies(hello ${${PROJECT_NAME}_EXPORTED_TARGETS}  
4 |     ${catkin_EXPORTED_TARGETS})  
5 |  
6 | target_link_libraries(hello  
7 |     ${catkin_LIBRARIES})
```

最后，编译并执行，控制台可以输出自定义的文本信息

3.2.2 自定义源文件调用

需求：设计头文件与源文件，在可执行文件中包含头文件。

流程：

1. 编写头文件；
2. 编写源文件；
3. 编写可执行文件；
4. 编辑配置文件并执行。

1.头文件

头文件设置于 3.2.1 类似，在功能包下的 include/功能包名 目录下新建头文件: haha.h，示例内容如下：

```
1 | #ifndef _HAHA_H  
2 | #define _HAHA_H  
3 |  
4 | namespace hello_ns {  
5 |  
6 | class My {  
7 |  
8 | public:  
9 |     void run();  
10 |  
11 | };  
12 |  
13 | }
```



```
14
15 #endif
```

注意:

在 VScode 中, 为了后续包含头文件时不抛出异常, 请配置 .vscode 下 c_cpp_properties.json 的 includepath 属性

```
1 | "/home/用户/工作空间/src/功能包/include/**"
```

2.源文件

在 src 目录下新建文件:haha.cpp, 示例内容如下:

```
1 | #include "test_head_src/haha.h"
2 | #include "ros/ros.h"
3 |
4 | namespace hello_ns{
5 |
6 | void My::run(){
7 |     ROS_INFO("hello,head and src ...");
8 | }
9 |
10 | }
```

3.可执行文件

在 src 目录下新建文件: use_head.cpp, 示例内容如下:

```
1 | #include "ros/ros.h"
2 | #include "test_head_src/haha.h"
3 |
4 | int main(int argc, char *argv[])
5 | {
6 |     ros::init(argc,argv,"hahah");
7 |     hello_ns::My my;
8 |     my.run();
9 |     return 0;
10 | }
```

4.配置文件

头文件与源文件相关配置:

```
1 | include_directories(
2 | include
3 |     ${catkin_INCLUDE_DIRS}
4 | )
5 |
6 | ## 声明C++库
7 | add_library(head
```

```

8   include/test_head_src/haha.h
9   src/haha.cpp
10  )
11
12  add_dependencies(head ${${PROJECT_NAME}_EXPORTED_TARGETS}
13                  ${catkin_EXPORTED_TARGETS})
14
15  target_link_libraries(head
16      ${catkin_LIBRARIES}
17  )

```

可执行文件配置:

```

1  add_executable(use_head src/use_head.cpp)
2
3  add_dependencies(use_head ${${PROJECT_NAME}_EXPORTED_TARGETS}
4                  ${catkin_EXPORTED_TARGETS})
5
6  #此处需要添加之前设置的 head 库
7  target_link_libraries(use_head
8      head
9      ${catkin_LIBRARIES}
10 )

```

3.3 Python模块导入

与C++类似的，在Python中导入其他模块时，也需要相关处理。

需求:首先新建一个Python文件A，再创建Python文件UseA，在UseA中导入A并调用A的实现。

实现:

1. 新建两个Python文件，使用 `import` 实现导入关系；
2. 添加可执行权限、编辑配置文件并执行UseA。

1.新建两个Python文件并使用import导入

文件A实现(包含一个变量):

```

1  #! /usr/bin/env python
2  num = 1000

```

文件B核心实现:

```
1 import os
2 import sys
3
4 path = os.path.abspath(".")
5 # 核心
6 sys.path.insert(0,path + "/src/plumbing_pub_sub/scripts")
7
8 import tools
9
10 ....
11 ....
12 rospy.loginfo("num = %d",tools.num)
```

2.添加可执行权限，编辑配置文件并执行

此过程略。

3.4 本章小结

本章内容相对比较简单，多加练习即可。

